

Sistema MLOps para Clasificación de Calidad de Vinos

Documentación técnica de un proyecto de Machine Learning Operations (MLOps) de extremo a extremo: entrenamiento, despliegue, monitoreo, observabilidad y tracking de experimentos

Campo	Detalle
Proyecto	Clasificación de Calidad de Vinos — Plataforma MLOps
Dataset	Wine Quality Dataset (UCI Machine Learning Repository)
Modelo en producción	Random Forest (serializado con Joblib)
Tipo de documento	Documentación técnica de arquitectura y operación
Despliegue	docker compose up --build
Estado	Activo

Universidad Tec Milenio

Carlos Alberto Damm Manzanera

07142372

Tabla de contenido

1. Introducción	3
2. Objetivos	4
2.1 Objetivo general	4
2.2 Objetivos específicos	4
3. Arquitectura del sistema	4
3.1 Diagrama de arquitectura	5
3.2 Principios de diseño	5
4. Tecnologías utilizadas	5
5. Descripción del modelo	6
5.1 Modelos comparados	6
5.2 Modelo seleccionado y serialización	7
6. Flujo de inferencia	7
7. API REST	7
7.1 Resumen de endpoints	7
7.2 Detalle de los endpoints	8
7.2.1 GET /health	8
7.2.2 POST /predict	8
7.2.3 GET /metrics	8
7.2.4 GET /drift	8
8. Monitoreo y observabilidad	9
8.1 Señales registradas	9
8.2 Logs estructurados	9
9. MLflow	9
9.1 Información registrada	9
9.2 Ejecución como servicio	10
9.3 Problemas resueltos durante la integración	10
9.3.1 Resolución de nombre de host dentro de Docker	10
9.3.2 Orden de inicialización de los servicios	10

10. Data Drift	10
11. Docker y Docker Compose	11
11.1 Servicios orquestados	11
11.2 Persistencia y healthchecks	11
12. Guía de instalación	12
12.1 Requisitos previos	12
12.2 Despliegue	12
13. Ejemplo de uso de la API	12
13.1 Verificación de estado	12
13.2 Solicitud de predicción	12
14. Resultados obtenidos	13
15. Conclusiones	14

1. Introducción

Este documento describe el diseño, la implementación y la operación de un sistema de **Machine Learning Operations (MLOps)** para la clasificación de la calidad de vinos. El proyecto nació originalmente como un ejercicio académico centrado exclusivamente en entrenar un modelo de Machine Learning capaz de clasificar la calidad de vinos utilizando el *Wine Quality Dataset* del *UCI Machine Learning Repository*.

Con el tiempo, el alcance evolucionó desde un único modelo entrenado hasta una plataforma completa de MLOps que cubre el ciclo de vida operativo del modelo: entrenamiento, empaquetado, despliegue, exposición mediante una API, monitoreo, observabilidad, tracking de experimentos y detección de desviaciones en los datos (*data drift*). El resultado es un sistema reproducible que puede levantarse por completo con un único comando.

El sistema, en su estado actual, integra los siguientes componentes:

- Modelo Random Forest entrenado previamente y serializado con Joblib.
- API REST desarrollada con FastAPI.
- Frontend en HTML/CSS/JavaScript.
- Nginx como servidor web.
- Docker y Docker Compose para la contenerización y orquestación.

- MLflow para el tracking de experimentos e inferencias.
- Monitoreo de métricas del sistema.
- Logs estructurados.
- Detección de Data Drift.
- Persistencia de experimentos.
- Healthchecks entre servicios.

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y documentar una plataforma de MLOps de extremo a extremo que exponga un modelo de clasificación de calidad de vinos como un servicio productivo, incorporando prácticas de despliegue reproducible, monitoreo, observabilidad, tracking de experimentos y detección de data drift.

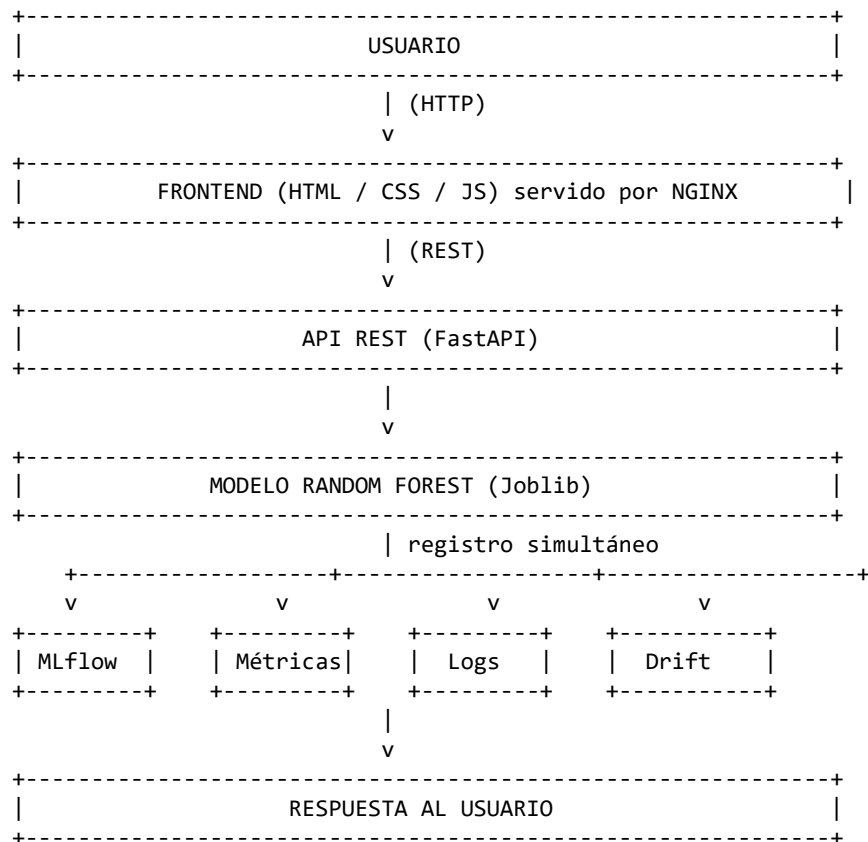
2.2 Objetivos específicos

- Entrenar y seleccionar un modelo de clasificación a partir de la comparación de varios algoritmos.
- Serializar el modelo seleccionado y cargarlo desde la capa de servicio.
- Exponer el modelo mediante una API REST con endpoints de salud, predicción, métricas y drift.
- Proporcionar una interfaz web para la interacción del usuario final.
- Registrar automáticamente las inferencias y sus métricas en MLflow.
- Implementar monitoreo de métricas operativas y logs estructurados.
- Incorporar un detector básico de data drift sobre las variables de entrada.
- Contenerizar todos los servicios y orquestarlos con Docker Compose.
- Garantizar un arranque ordenado de los servicios mediante healthchecks y dependencias.
- Permitir el despliegue completo del sistema con un único comando.

3. Arquitectura del sistema

La arquitectura del sistema sigue un patrón de servicios desacoplados, orquestados mediante Docker Compose. El usuario interactúa con un frontend estático servido por Nginx, el cual comunica con la API REST de FastAPI. La API carga el modelo Random Forest y, en cada inferencia, registra información de forma simultánea en MLflow, en el subsistema de métricas, en los logs estructurados y en el detector de data drift.

3.1 Diagrama de arquitectura



3.2 Principios de diseño

- **Servicios desacoplados:** el frontend, la API y MLflow se ejecutan como contenedores independientes.
- **Reproducibilidad:** todo el entorno se define de forma declarativa y se levanta con un único comando.
- **Observabilidad integrada:** cada inferencia genera métricas, logs y registros de tracking.
- **Arranque ordenado:** las dependencias entre servicios se garantizan mediante healthchecks.
- **Persistencia:** los experimentos y artefactos de MLflow se conservan en volúmenes.

4. Tecnologías utilizadas

La siguiente tabla resume las tecnologías que componen el sistema y el rol que desempeña cada una dentro de la plataforma MLOps.

Tecnología	Categoría	Rol en el sistema
Random Forest	Modelo de ML	Algoritmo seleccionado para la clasificación de la calidad del vino.
Joblib	Serialización	Persistencia del modelo entrenado para su posterior carga.
FastAPI	Backend / API	Exposición del modelo mediante una API REST.
HTML / CSS / JavaScript	Frontend	Interfaz web para la interacción del usuario final.
Nginx	Servidor web	Servido del frontend estático.
Docker	Contenerización	Empaquetado de cada servicio en contenedores.
Docker Compose	Orquestación	Definición y arranque coordinado de los servicios.
MLflow	Tracking	Registro de inferencias, parámetros, métricas y experimentos.
SQLite	Backend de MLflow	Almacén de metadatos de los experimentos de MLflow.

Supuesto: las categorías indicadas en la tabla se incluyen con fines organizativos y no implican dependencias o versiones específicas más allá de las descritas en este documento.

5. Descripción del modelo

El modelo tiene como objetivo clasificar la calidad de vinos a partir de sus variables fisicoquímicas, utilizando el *Wine Quality Dataset* del UCI Machine Learning Repository. Durante la fase de desarrollo se compararon tres algoritmos de clasificación con el fin de seleccionar el que ofreciera el mejor desempeño y estabilidad.

5.1 Modelos comparados

Algoritmo	Accuracy	F1 Score	ROC-AUC	Estabilidad (CV)
Logistic Regression	Menor	Menor	Menor	Menor
Decision Tree	Intermedio	Intermedio	Intermedio	Menor
Random Forest	Mejor	Mejor	Mejor	Mejor

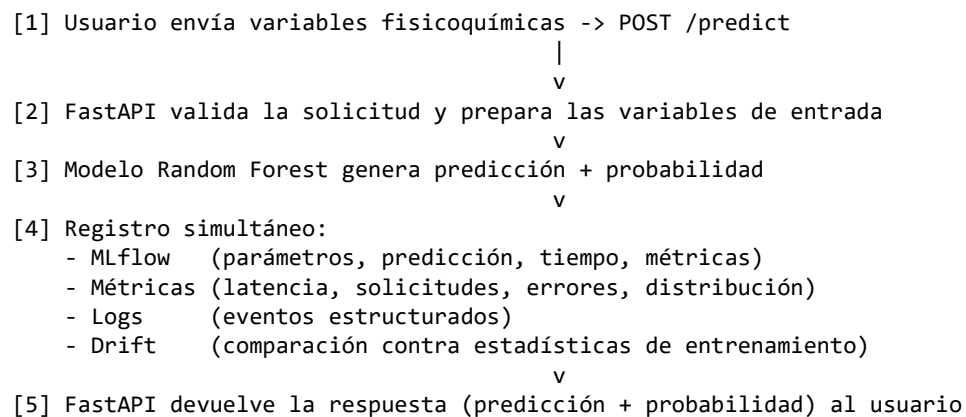
Supuesto: la tabla presenta la comparación de forma cualitativa (relativa entre modelos), ya que este documento no fija valores numéricos concretos para las métricas. Random Forest resultó ser el modelo con el mejor desempeño en Accuracy, F1 Score, ROC-AUC y estabilidad en validación cruzada.

5.2 Modelo seleccionado y serialización

El modelo seleccionado fue **Random Forest**, debido a su mejor desempeño en Accuracy, F1 Score y ROC-AUC, así como a su mayor estabilidad durante la validación cruzada. Una vez entrenado, el modelo se serializó mediante **Joblib** y, posteriormente, se cargó desde la capa de servicio (FastAPI) para su uso en producción. De esta forma, el proceso de entrenamiento y el de servicio quedan desacoplados: la API consume un artefacto ya entrenado en lugar de reentrenar en tiempo de ejecución.

6. Flujo de inferencia

El flujo de inferencia describe el recorrido de una solicitud de predicción desde el usuario hasta la respuesta final. Cada solicitud recibida por el endpoint de predicción desencadena el registro simultáneo de información en MLflow, en el subsistema de métricas, en los logs estructurados y en el detector de data drift.



7. API REST

La API fue desarrollada con **FastAPI** y constituye la capa de servicio que expone el modelo. Proporciona endpoints para verificar el estado del servicio, generar predicciones, consultar métricas del sistema y obtener información del detector de data drift.

7.1 Resumen de endpoints

Método	Ruta	Descripción
--------	------	-------------

GET	/health	Verifica que la API esté disponible y que el modelo se haya cargado correctamente.
POST	/predict	Recibe las variables fisicoquímicas del vino y devuelve la predicción y su probabilidad.
GET	/metrics	Devuelve métricas del sistema: número de solicitudes, errores, latencia y distribución de predicciones.
GET	/drift	Devuelve información del detector de Data Drift.

7.2 Detalle de los endpoints

7.2.1 GET /health

Endpoint de verificación de estado (*healthcheck*). Confirma dos condiciones esenciales para operar:

- Que la API se encuentre disponible.
- Que el modelo se haya cargado correctamente.

7.2.2 POST /predict

Endpoint principal de inferencia. Recibe las variables fisicoquímicas del vino y devuelve:

- La predicción generada por el modelo Random Forest.
- La probabilidad asociada a dicha predicción.

Cada llamada a este endpoint desencadena, de forma simultánea, el registro en MLflow, la actualización de métricas, la generación de logs estructurados y la evaluación del detector de data drift.

7.2.3 GET /metrics

Devuelve métricas operativas del sistema, entre ellas:

- Número de solicitudes.
- Número de errores.
- Latencia.
- Distribución de predicciones.

7.2.4 GET /drift

Devuelve la información generada por el detector de Data Drift, permitiendo consultar el estado de la comparación entre los datos recibidos en las inferencias y las estadísticas obtenidas durante el entrenamiento.

8. Monitoreo y observabilidad

El sistema incorpora capacidades de monitoreo y observabilidad que permiten conocer el estado operativo del servicio y el comportamiento del modelo en tiempo de ejecución. La información recolectada se expone a través del endpoint **/metrics** y se complementa con logs estructurados.

8.1 Señales registradas

Señal	Descripción
Tiempo de inferencia	Tiempo requerido por el modelo para generar una predicción.
Latencia promedio	Latencia media observada en el servicio.
Número de solicitudes	Total de solicitudes atendidas por la API.
Número de errores	Total de errores registrados durante la operación.
Distribución de predicciones	Distribución de las clases predichas por el modelo.
Eventos (logs)	Eventos registrados mediante logs estructurados.

8.2 Logs estructurados

Los eventos del sistema se registran mediante logs estructurados, lo que facilita su lectura, filtrado y análisis posterior. Estos registros complementan las métricas numéricas y aportan trazabilidad sobre la operación del servicio y sobre los eventos relevantes generados durante las inferencias.

9. MLflow

MLflow se integró en la plataforma para registrar automáticamente las inferencias realizadas por la API. Cada solicitud atendida por el endpoint de predicción genera información de tracking que permite auditar y analizar el comportamiento del modelo a lo largo del tiempo.

9.1 Información registrada

- Parámetros de la solicitud.
- Predicción generada.
- Tiempo de respuesta.
- Métricas asociadas.

- Experimento correspondiente.

9.2 Ejecución como servicio

MLflow se ejecuta como un servicio independiente dentro de Docker Compose. Utiliza **SQLite** como backend para el almacenamiento de metadatos y dispone de almacenamiento persistente para los artefactos, de modo que los experimentos se conservan entre reinicios de los contenedores.

9.3 Problemas resueltos durante la integración

9.3.1 Resolución de nombre de host dentro de Docker

Inicialmente, la API intentaba conectarse a MLflow mediante la dirección **http://127.0.0.1:5000**. Sin embargo, dentro de un contenedor, esa dirección apunta al propio contenedor y no al servicio de MLflow. La solución consistió en utilizar el nombre del servicio definido en Docker Compose como nombre de host, empleando la dirección **http://mlflow:5000**, que la red interna de Docker Compose resuelve correctamente hacia el contenedor de MLflow.

9.3.2 Orden de inicialización de los servicios

Posteriormente se identificó un problema de inicialización: la API arrancaba antes que MLflow, lo que provocaba fallos de conexión al no estar disponible el servicio de tracking. La solución consistió en implementar un **healthcheck** para MLflow y declarar la dependencia mediante **depends_on** con la condición **service_healthy**. De esta forma, FastAPI únicamente inicia cuando MLflow ya se encuentra disponible y en estado saludable.

10. Data Drift

El proyecto incorpora un detector básico de **Data Drift** cuyo objetivo es identificar cambios en la distribución de los datos de entrada que puedan degradar el desempeño del modelo con el paso del tiempo. Cada solicitud recibida se compara contra las estadísticas obtenidas durante el entrenamiento.

Cuando una variable supera los límites esperados, el sistema registra un evento que incluye la siguiente información:

Campo	Descripción
Variable	Variable fisicoquímica que superó los límites esperados.

Valor recibido	Valor de la variable en la solicitud analizada.
Media	Media de la variable obtenida durante el entrenamiento.
Desviación estándar	Desviación estándar de la variable obtenida durante el entrenamiento.

Este mecanismo permite detectar de forma temprana desviaciones en la distribución de los datos, aportando una señal útil para el mantenimiento del modelo y para decidir eventuales acciones correctivas. La información del detector puede consultarse mediante el endpoint **/drift**.

11. Docker y Docker Compose

Todo el proyecto fue contenerizado con **Docker** y orquestado mediante **Docker Compose**. Este enfoque garantiza un entorno reproducible y un arranque coordinado de todos los servicios con un único comando.

11.1 Servicios orquestados

Servicio	Función	Notas de operación
Frontend	Interfaz web (HTML/CSS/JS) servida por Nginx.	Punto de entrada del usuario.
FastAPI	API REST que expone el modelo.	Inicia únicamente cuando MLflow está saludable (depends_on: service_healthy).
MLflow	Servicio de tracking de experimentos.	Backend SQLite y almacenamiento persistente de artefactos; expone un healthcheck.

11.2 Persistencia y healthchecks

Se utilizan volúmenes persistentes para conservar los experimentos y artefactos de MLflow, de manera que la información no se pierda al reiniciar los contenedores.

Adicionalmente, los **healthchecks** entre servicios garantizan un arranque ordenado:

FastAPI espera a que MLflow esté disponible y saludable antes de iniciar, evitando fallos de conexión durante el arranque.

12. Guía de instalación

El sistema completo puede desplegarse con un único comando gracias a Docker Compose. A continuación, se describe el procedimiento general de instalación y despliegue.

12.1 Requisitos previos

- Docker instalado en el equipo.
- Docker Compose disponible.

Supuesto: se asume que el código fuente del proyecto ya se encuentra disponible localmente y que el equipo dispone de acceso a Docker y Docker Compose. Este documento no especifica versiones concretas de estas herramientas.

12.2 Despliegue

Desde el directorio raíz del proyecto, ejecutar el siguiente comando para construir las imágenes e iniciar todos los servicios:

```
docker compose up --build
```

Docker Compose construirá e iniciará automáticamente el frontend, la API de FastAPI y MLflow. Gracias a los healthchecks y a la dependencia declarada, FastAPI iniciará únicamente cuando MLflow se encuentre disponible.

13. Ejemplo de uso de la API

A continuación se muestra un ejemplo conceptual de interacción con la API. El endpoint de predicción recibe las variables fisicoquímicas del vino y devuelve la predicción junto con su probabilidad.

13.1 Verificación de estado

```
GET /health
```

Devuelve la confirmación de que la API está disponible y de que el modelo se ha cargado correctamente.

13.2 Solicitud de predicción

```
POST /predict
Content-Type: application/json

{
```

```
"features": { ...variables fisicoquímicas del vino... }
}
```

La respuesta contiene la predicción del modelo y la probabilidad asociada:

```
{
  "prediction": ...,
  "probability": ...
}
```

14. Resultados obtenidos

El proyecto alcanzó su objetivo de transformar un modelo de Machine Learning aislado en una plataforma de MLOps funcional y operable. Los resultados obtenidos, de acuerdo con el alcance implementado, se resumen a continuación.

- El sistema completo se despliega y ejecuta correctamente con un único comando (docker compose up --build).
- La API de FastAPI expone el modelo Random Forest y responde a los endpoints de salud, predicción, métricas y drift.
- El modelo seleccionado (Random Forest) se carga desde su artefacto serializado con Joblib, sin necesidad de reentrenar en tiempo de ejecución.
- Cada inferencia queda registrada de forma simultánea en MLflow, en el subsistema de métricas, en los logs estructurados y en el detector de data drift.
- MLflow se ejecuta como servicio independiente con backend SQLite y almacenamiento persistente, conservando los experimentos entre reinicios.
- El detector básico de data drift identifica y registra las variables que superan los límites esperados respecto a las estadísticas de entrenamiento.
- Los problemas de integración detectados (resolución de host y orden de arranque) se resolvieron mediante el uso del nombre de servicio de Docker Compose y de healthchecks con depends_on.

Objetivo	Resultado
Despliegue reproducible con un solo comando	Logrado mediante Docker y Docker Compose.
Exposición del modelo como servicio	Logrado con una API REST en FastAPI y sus cuatro endpoints.
Tracking de inferencias	Logrado con MLflow como servicio persistente.
Monitoreo y observabilidad	Logrado mediante métricas del sistema y logs estructurados.

Detección de data drift	Logrado con un detector básico sobre las variables de entrada.
Arranque ordenado de servicios	Logrado con healthchecks y depends_on (condition: service_healthy).

Supuesto: la tabla resume el cumplimiento de los objetivos en términos funcionales (logrado / no logrado). Este documento no reporta valores numéricos concretos de métricas como accuracy, latencia o número de solicitudes, ya que dependen de la ejecución y los datos utilizados en cada corrida.

15. Conclusiones

El proyecto cumplió su propósito: partiendo de un ejercicio centrado exclusivamente en entrenar un modelo de clasificación de calidad de vinos, evolucionó hacia una plataforma de MLOps completa y funcional. El sistema no solo entrena y sirve un modelo, sino que también lo despliega, lo monitorea, lo observa, registra sus experimentos y vigila la estabilidad de sus datos de entrada, todo ello de forma reproducible y con un único comando.

Más allá del resultado técnico, el proyecto demostró en la práctica cómo implementar un ciclo de MLOps y cuáles son sus beneficios principales:

- **Reproducibilidad:** cualquier persona puede levantar el sistema completo con un único comando, eliminando la fricción de configuración manual.
- **Desacoplamiento entre entrenamiento y servicio:** el modelo se entrena una vez, se serializa y se consume como artefacto, lo que simplifica la operación.
- **Observabilidad:** las métricas y los logs estructurados aportan visibilidad sobre el comportamiento del modelo y del servicio en tiempo de ejecución.
- **Trazabilidad:** MLflow permite auditar las inferencias, sus parámetros y sus resultados a lo largo del tiempo.
- **Mantenimiento proactivo:** la detección de data drift ofrece una señal temprana ante posibles degradaciones del modelo.
- **Robustez operativa:** los healthchecks y las dependencias entre servicios garantizan un arranque ordenado y confiable.

En conjunto, la experiencia validó que las prácticas de MLOps aportan valor incluso a un proyecto de alcance académico, al llevar un modelo desde la fase experimental hasta un servicio operable, monitoreado y mantenible. Como líneas de trabajo futuro —*supuesto, no implementado en el alcance actual*— podrían considerarse la incorporación de

reentrenamiento automatizado, la ampliación del detector de data drift con pruebas estadísticas más formales y la integración de un pipeline de CI/CD.